Licence MIASHS deuxième année

Rapport de Projet Web Dynamique

Site de Gestion d'Association

Projet réalisé du 30 Janvier 2026 au 30 Avril 2026

Membres du groupe

TEDJIOGNY Fabrice (42014524)

`https://github.com/FabrTED/coherence-deputes`

Remerciements

Nous tenons tout d'abord à exprimer notre profonde gratitude envers nos enseignants, qui nous ont accompagnés et guidés tout au long de la réalisation de ce projet. Leurs conseils, leur disponibilité et leur pédagogie ont été essentiels pour nous permettre de mener à bien ce travail dans de bonnes conditions.

Nous souhaitons également remercier les différents outils numériques utilisés au cours de ce projet, notamment les ressources en ligne et les assistants d'intelligence artificielle tels que ChatGPT d'OpenAI. Ces outils nous ont permis de mieux comprendre certains concepts techniques liés au développement web, d'approfondir nos connaissances et de surmonter plusieurs difficultés rencontrées durant le projet.

Enfin, nous remercions toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet. Leur soutien et leur aide ont été précieux et ont largement participé à la réussite de ce travail.

Table des matières

1 Introduction **4**

2 Environnement de travail **4**

3 Utilisation de l'intelligence artificielle **5**
3.1 Aide apportée par l'IA dans la réalisation du projet 5
3.2 Fonctionnement général des outils d'IA utilisés 5

4 Description du site **6**

5 Base de données **6**
5.1 Table `membre` 7
5.2 Table `activite` 7
5.3 Table `membre_activite` 7

6 Fonctionnalités principales **7**
6.1 Gestion des membres 7
6.2 Gestion des activités 7
6.3 Gestion des inscriptions 8
6.4 Interface utilisateur 8
6.5 Recherche dynamique 8
6.6 Sécurité 8

7 Difficultés rencontrées **8**
7.1 Gestion des interactions entre PHP et la base de données 8
7.2 Gestion des formulaires et des actions 9
7.3 Gestion des doublons 9
7.4 JavaScript et interactions dynamiques 9
7.5 Organisation du projet 9

8 Bilan, conclusion et perspectives **9**
8.1 Bilan 9
8.2 Conclusion 10
8.3 Perspectives 10

9 Bibliographie **11**

10 Webographie **12**

11 Annexes **13**
11.1 Cahier des charges 13
11.2 Manuel utilisateur 13
11.3 Structure du projet 13
11.4 Structure de la base de données 14
11.5 Extraits de code PHP 14
11.6 Script JavaScript (recherche dynamique) 14
11.7 Captures supplémentaires 15

1 Introduction

Dans le cadre de notre projet de **Deuxième Année (L2)**, nous avons développé un **site web dynamique** ayant pour objectif la gestion d'une association. Ce projet vise à faciliter l'administration des **membres** ainsi que l'organisation et le suivi des **activités** proposées par l'association.

Le site a été conçu en s'appuyant sur plusieurs technologies complémentaires. La structure et l'interface utilisateur reposent sur **HTML** et **CSS**, tandis que **JavaScript** permet d'ajouter des interactions dynamiques, notamment pour la recherche en temps réel et la confirmation des actions. La partie serveur est assurée par **PHP**, qui permet le traitement des formulaires, la gestion des sessions et la communication avec la base de données. Enfin, les données sont stockées dans une base **MySQL**, administrée via **PhpMyAdmin**, dans un environnement local fourni par **XAMPP**.

L'application permet de réaliser les principales opérations de gestion, telles que l'**ajout**, la **modification** et la **suppression** des membres et des activités. Elle permet également d'**inscrire un membre à une activité** et de gérer ces inscriptions. Pour cela, une relation de type **plusieurs-à-plusieurs** a été mise en place entre les membres et les activités, grâce à une table de liaison dédiée.

Afin d'assurer la **cohérence des données** et d'éviter les erreurs, des mécanismes de contrôle ont été intégrés. Par exemple, chaque membre possède un **email unique**, chaque activité un **titre unique**, et une même inscription (membre + activité) ne peut être enregistrée qu'une seule fois. De plus, une fonctionnalité de **recherche dynamique** a été développée pour améliorer l'expérience utilisateur et faciliter la navigation.

Ce projet nous a permis de mettre en pratique les notions essentielles du **développement web dynamique**, notamment la gestion des bases de données, le traitement des requêtes, et l'interaction entre le client et le serveur, tout en développant une application concrète et fonctionnelle.

2 Environnement de travail

Le développement de ce site web dynamique a été réalisé sur un environnement local à l'aide de **XAMPP**, permettant d'exécuter un serveur **Apache** ainsi qu'une base de données **MySQL**. Le projet a été développé sous un système **Windows**, offrant un cadre stable et adapté au développement web.

Le site repose principalement sur le langage **PHP**, utilisé pour la gestion de la logique côté serveur (traitement des formulaires, interactions avec la base de données, gestion des sessions). La structure et l'affichage des pages sont assurés par **HTML** et **CSS**, tandis que **JavaScript** est utilisé pour améliorer l'interactivité, notamment pour la **recherche dynamique** et les **confirmations d'actions** (suppression, modification).

La gestion de la base de données a été réalisée avec **phpMyAdmin**, permettant de créer et manipuler les tables telles que **membre**, **activite** et **membre_activite**, tout en facilitant le test des requêtes SQL et la vérification des données.

Pour le développement du code, l'environnement **Visual Studio Code** a été utilisé.

Cet outil permet une meilleure lisibilité du code grâce à la **coloration syntaxique**, l'**auto-indentation** et une organisation claire des fichiers. Il facilite également la navigation entre les différentes pages du projet.

Enfin, des phases de test ont été réalisées directement sur le navigateur en local (localhost), afin de vérifier le bon fonctionnement des fonctionnalités principales : **ajout, modification, suppression, inscription et désinscription**, ainsi que la gestion des cas particuliers comme les **doublons** ou les erreurs utilisateur.

3 Utilisation de l'intelligence artificielle

3.1 Aide apportée par l'IA dans la réalisation du projet

Dans le cadre de ce projet, des outils d'intelligence artificielle ont été utilisés comme **support méthodologique**, sans remplacer le travail de conception et de développement.

L'IA a permis notamment de :
- Clarifier certains concepts liés au développement web (**PHP, SQL, interactions client-serveur**)
- Aider à structurer le code et organiser les différentes pages du site
- Identifier et corriger des erreurs techniques (requêtes SQL, logique PHP, gestion des formulaires)
- Améliorer la lisibilité et la qualité du code (indentation, bonnes pratiques)
- Faciliter la compréhension globale du projet et de ses différentes fonctionnalités

L'IA a également été utilisée pour améliorer la rédaction du rapport, notamment en reformulant certaines explications et en structurant les différentes parties de manière plus claire.

Cependant, toutes les décisions techniques, les choix de conception et les validations finales ont été réalisés de manière autonome. L'intelligence artificielle a donc joué un rôle **d'assistant pédagogique**, et non de générateur automatique de solution

3.2 Fonctionnement général des outils d'IA utilisés

Les outils d'intelligence artificielle utilisés, notamment **ChatGPT (OpenAI)**, reposent sur des modèles appelés **modèles de langage** capables d'analyser et de générer du texte en fonction du contexte.

Ces modèles sont entraînés sur de grandes quantités de données et fonctionnent en prédisant les mots les plus pertinents à partir d'une demande. Ils sont ainsi capables de fournir des explications, proposer des corrections ou suggérer des améliorations de code.

Dans ce projet, l'IA n'a pas été intégrée directement dans le site web (pas de fonctionnalité IA côté utilisateur). Elle a uniquement été utilisée **en amont du développement**, comme outil d'assistance pour :

— la compréhension des concepts
— la résolution de problèmes techniques
— l'optimisation du code

Il est important de noter que les réponses fournies par ces outils ne sont pas toujours exactes. Elles ont donc été **analysées, adaptées et vérifiées** avant d'être intégrées dans le projet.

Ainsi, l'IA a contribué à rendre le développement plus efficace, tout en nécessitant un **esprit critique** pour garantir la qualité et la cohérence du résultat final.

4 Description du site

Le projet consiste en la réalisation d'un **site web dynamique de gestion d'association**, permettant d'administrer des membres et des activités, ainsi que de gérer les inscriptions.

Le site distingue deux types d'utilisateurs :
— **Administrateur** : accès complet à la gestion du site
— **Utilisateur** : accès aux activités et à ses inscriptions

Le fonctionnement global repose sur une interaction entre :
— le **frontend** (HTML, CSS, JavaScript)
— le **backend** (PHP)
— la **base de données** (MySQL)

L'administrateur peut :
— Ajouter, modifier et supprimer des membres
— Ajouter, modifier et supprimer des activités
— Inscrire un membre à une activité
— Désinscrire un membre d'une activité
— Visualiser les membres inscrits

L'utilisateur peut :
— Consulter les activités disponibles
— S'inscrire à une activité
— Consulter ses inscriptions

Le site utilise également **JavaScript** pour améliorer l'expérience utilisateur, notamment avec la **recherche dynamique** et les **confirmations d'actions**.

5 Base de données

Le site repose sur une base de données relationnelle composée de trois tables principales.

5.1 Table `membre`

Cette table contient les informations des membres :
— id (clé primaire)
— nom
— prenom
— email
— date_inscription
Une contrainte a été mise en place pour garantir un **email unique**.

5.2 Table `activite`

Cette table contient les activités :
— id (clé primaire)
— titre
— description
— date_activite
Une contrainte empêche la duplication des **titres d'activités**.

5.3 Table `membre_activite`

Cette table permet de gérer les inscriptions entre membres et activités.
— id_membre
— id_activite
Elle représente une relation **many-to-many**. Une contrainte UNIQUE empêche une **double inscription** d'un membre à une même activité.

6 Fonctionnalités principales

6.1 Gestion des membres

L'administrateur peut :
— Ajouter un membre
— Modifier ses informations
— Supprimer un membre
— Inscrire un membre à une activité
Un contrôle empêche la création de **doublons d'emails**.

6.2 Gestion des activités

L'administrateur peut :
— Ajouter une activité
— Modifier une activité
— Supprimer une activité
— Visualiser les membres inscrits
Un contrôle empêche les **doublons de titres**.

6.3 Gestion des inscriptions

Le site permet :
— D'inscrire un membre à une activité
— De désinscrire un membre depuis la page activité
Un système empêche la **double inscription**.

6.4 Interface utilisateur

Le site propose une interface simple et claire :
— Menu de navigation
— Tableaux organisés
— Formulaires intuitifs
Le design est réalisé en **CSS**.

6.5 Recherche dynamique

Une fonctionnalité en **JavaScript** permet de filtrer en temps réel :
— Les membres
— Les activités
Cette fonctionnalité améliore fortement l'expérience utilisateur.

6.6 Sécurité

Plusieurs mécanismes de sécurité ont été mis en place :
— Requêtes préparées (**PDO**) pour éviter les injections SQL
— Validation des données côté serveur
— Confirmations JavaScript avant actions critiques

7 Difficultés rencontrées

Au cours de la réalisation de ce projet, plusieurs difficultés techniques et organisationnelles ont été rencontrées.

7.1 Gestion des interactions entre PHP et la base de données

L'une des principales difficultés a concerné la gestion des échanges entre le code **PHP** et la base de données **MySQL**. Il a notamment été nécessaire de bien comprendre le fonctionnement des **requêtes préparées (PDO)** ainsi que la récupération et le traitement des données.

Des erreurs fréquentes ont été observées au niveau des requêtes SQL (mauvaises jointures, erreurs de syntaxe, incohérences dans les données), nécessitant plusieurs phases de test et de correction.

7.2 Gestion des formulaires et des actions

La mise en place des formulaires (ajout, modification, inscription) a également posé des difficultés. Il a fallu bien distinguer les différentes actions via la méthode `POST` et gérer correctement les conditions pour éviter les conflits entre les traitements.

Par exemple, la distinction entre les actions `add`, `edit` et `inscrire` a demandé une bonne organisation du code afin d'éviter des comportements inattendus.

7.3 Gestion des doublons

Un point particulièrement complexe a été la gestion des **doublons** :
— empêcher deux membres d'avoir le même email
— empêcher une double inscription à une activité
— empêcher deux activités avec le même titre

Cela a nécessité la mise en place de **vérifications côté PHP** ainsi que l'utilisation de **contraintes SQL** pour garantir la cohérence des données.

7.4 JavaScript et interactions dynamiques

L'ajout de fonctionnalités en **JavaScript**, comme la recherche dynamique, a également été une source de difficulté. Il a fallu comprendre comment manipuler le **DOM** pour filtrer les éléments en temps réel sans recharger la page.

Des problèmes d'affichage ont été rencontrés, notamment la disparition de certaines parties du tableau lors de la recherche, ce qui a nécessité des ajustements précis dans le code JavaScript.

7.5 Organisation du projet

Enfin, l'organisation globale du projet (répartition des fichiers, séparation entre logique et affichage, gestion des pages) a demandé une réflexion importante afin de garder un code clair et maintenable.

8 Bilan, conclusion et perspectives

8.1 Bilan

La réalisation de ce projet a permis de mettre en pratique de manière concrète les connaissances acquises en développement web et en gestion de bases de données.

Sur le plan technique, ce travail a permis de consolider l'utilisation du langage **PHP** pour la gestion de la logique côté serveur, ainsi que l'interaction avec une base de données **MySQL**. L'utilisation de **requêtes préparées (PDO)** a également permis de mieux comprendre les enjeux liés à la sécurité, notamment la protection contre les injections SQL.

Le projet a également mobilisé des compétences en **HTML** et **CSS** pour la structuration et la mise en forme des pages, ainsi qu'en **JavaScript** pour améliorer l'interactivité, en particulier avec la mise en place de la recherche dynamique et des confirmations d'actions.

Par ailleurs, ce projet a permis de mieux appréhender des notions importantes telles que la **validation des données**, la **gestion des erreurs**, ainsi que la **cohérence des informations**, notamment à travers la gestion des doublons (email unique, inscription unique, activité unique).

Enfin, il a mis en évidence l'importance d'une bonne organisation du code et d'une approche progressive basée sur des phases de test régulières.

8.2 Conclusion

Ce projet de site web dynamique de gestion d'association a permis de développer une application fonctionnelle répondant aux objectifs fixés. Les principales fonctionnalités ont été mises en place avec succès : gestion des membres, gestion des activités, inscription et désinscription, ainsi que visualisation des données.

Malgré certaines difficultés techniques rencontrées, notamment dans la gestion des formulaires, des interactions avec la base de données et des comportements dynamiques en JavaScript, le projet a pu être mené à terme grâce à une démarche rigoureuse de test et de correction.

Ce travail constitue ainsi une première expérience complète de développement d'un site web dynamique, combinant plusieurs technologies et nécessitant une bonne coordination entre le frontend et le backend.

8.3 Perspectives

Plusieurs améliorations pourraient être envisagées pour enrichir le projet et le rendre plus complet.

Dans un premier temps, il serait possible d'améliorer l'interface utilisateur en proposant un design plus avancé, par exemple à l'aide d'un framework CSS tel que **Bootstrap**, afin d'obtenir une interface plus moderne et responsive.

Ensuite, des fonctionnalités supplémentaires pourraient être ajoutées, comme :
— un système de **comptes utilisateurs** avec gestion des rôles plus avancée
— l'envoi de **notifications ou emails** lors des inscriptions
— une interface permettant aux utilisateurs de modifier leurs propres informations

Il serait également pertinent d'améliorer la partie base de données, en ajoutant davantage de contraintes et en optimisant certaines requêtes pour de meilleures performances.

Enfin, une évolution vers une architecture plus structurée (séparation complète entre logique métier et affichage, utilisation de frameworks) permettrait de rendre le projet plus professionnel et maintenable à long terme.

Ainsi, ce projet constitue une base solide pouvant être étendue vers des applications web plus complexes.

9 Bibliographie

[label] Auteur, TITRE, editeur, annee

[LAM94] L. LAMPORT, *LaTeX : A Document preparation system, Addison-Wesley, 1994*

10 Webographie

[1] **Documentation PHP**
https://www.php.net/

[2] **Documentation MySQL**
https://dev.mysql.com/doc/

[3] **W3Schools - Web Development Tutorials**
https://www.w3schools.com/

[4] **MDN Web Docs**
https://developer.mozilla.org/

[5] **JavaScript Guide (MDN)**
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

[6] **PHP PDO Documentation**
https://www.php.net/manual/fr/book.pdo.php

[7] **XAMPP Documentation**
https://www.apachefriends.org/index.html

[8] **phpMyAdmin Documentation**
https://www.phpmyadmin.net/

11 Annexes

11.1 Cahier des charges

L'objectif du projet est de développer un **site web dynamique de gestion d'association** permettant d'administrer des membres et des activités.

Le site doit permettre :
— la gestion des membres (ajout, modification, suppression)
— la gestion des activités
— l'inscription et la désinscription des membres aux activités
— la consultation des inscriptions

Le système doit garantir :
— l'absence de doublons (email unique, inscription unique)
— la sécurité des données (requêtes préparées)
— une interface simple et fonctionnelle

11.2 Manuel utilisateur

Administrateur :

L'administrateur accède à une interface complète lui permettant de gérer le site.
— Accéder au tableau de bord après connexion
— Ajouter un membre en remplissant le formulaire dédié
— Modifier ou supprimer un membre via les actions disponibles dans le tableau
— Créer une activité (titre, description, date)
— Modifier ou supprimer une activité existante
— Inscrire un membre à une activité via le bouton *Inscrire*
— Désinscrire un membre directement depuis la page des activités
— Utiliser la barre de recherche pour filtrer rapidement les membres ou activités

Utilisateur :

L'utilisateur dispose d'une interface simplifiée.
— Consulter la liste des activités disponibles
— Utiliser la barre de recherche pour trouver une activité
— S'inscrire à une activité en cliquant sur le bouton *Inscrire*
— Accéder à la page *Mes inscriptions*
— Consulter les activités auxquelles il est inscrit

11.3 Structure du projet

```
association/

 activite.php
 activites_public.php
 membre.php
 mes_inscriptions.php
 login.php
 logout.php
 index.php
```

```
config.php
header.php
footer.php

script.js
style.css
```

11.4 Structure de la base de données

Table **membre** :
- id (PRIMARY KEY)
- nom
- prenom
- email (UNIQUE)

Table **activite** :
- id (PRIMARY KEY)
- titre (UNIQUE)
- description
- date_activite

Table **membre_activite** :
- id_membre
- id_activite
- UNIQUE(id_membre, id_activite)

11.5 Extraits de code PHP

Insertion d'une inscription :

```
$stmt = $pdo->prepare("
    INSERT INTO membre_activite (id_membre, id_activite)
    VALUES (?, ?)
");
$stmt->execute([$id_membre, $id_activite]);
```

Vérification des doublons :

```
$check = $pdo->prepare("
    SELECT COUNT(*) FROM membre_activite
    WHERE id_membre = ? AND id_activite = ?
");
$check->execute([$id_membre, $id_activite]);
```

11.6 Script JavaScript (recherche dynamique)

```
const input = document.getElementById("search");

input.addEventListener("keyup", function () {
    const value = this.value.toLowerCase();

    document.querySelectorAll("table tbody tr").forEach(row => {
        const text = row.textContent.toLowerCase();
```

```
        row.style.display = text.includes(value) ? "" : "none";
    });
});
```

11.7 Captures supplémentaires

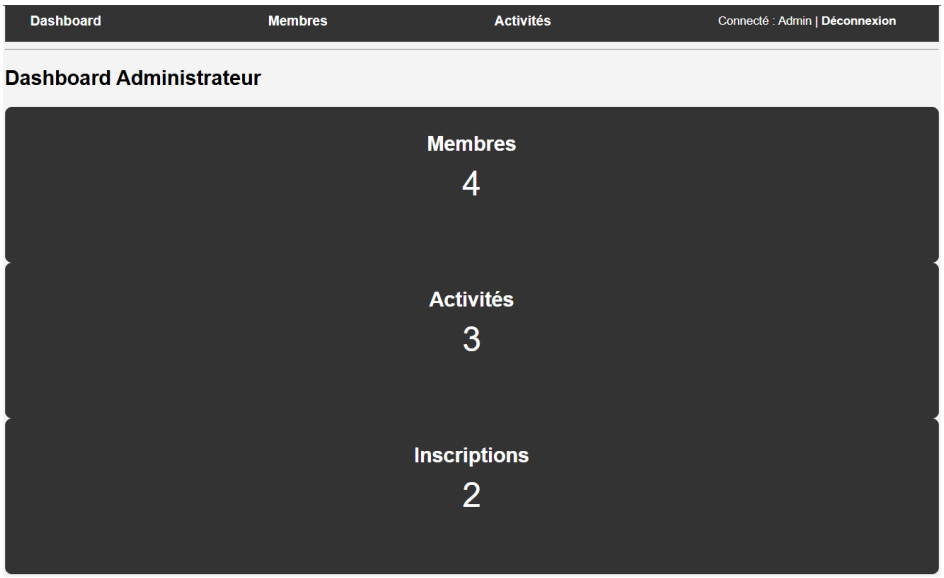

FIGURE 1 – Interface principale administrateur

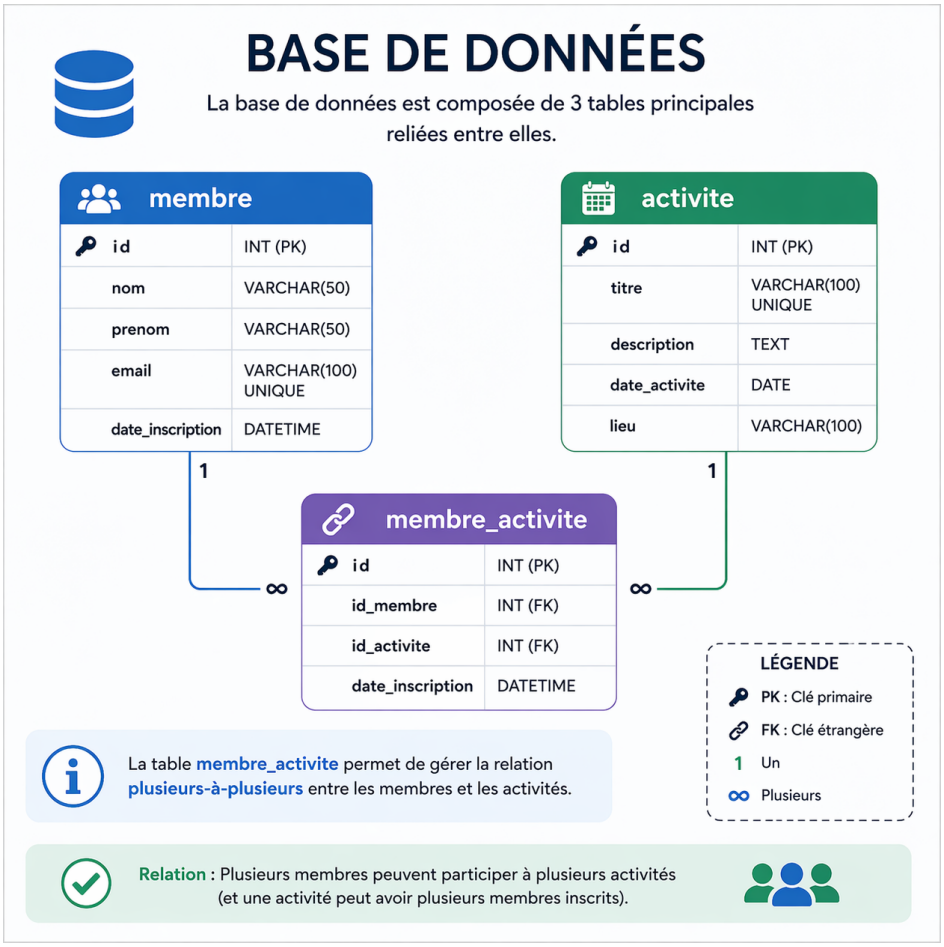

FIGURE 2 – Schéma relationnel de la base de données

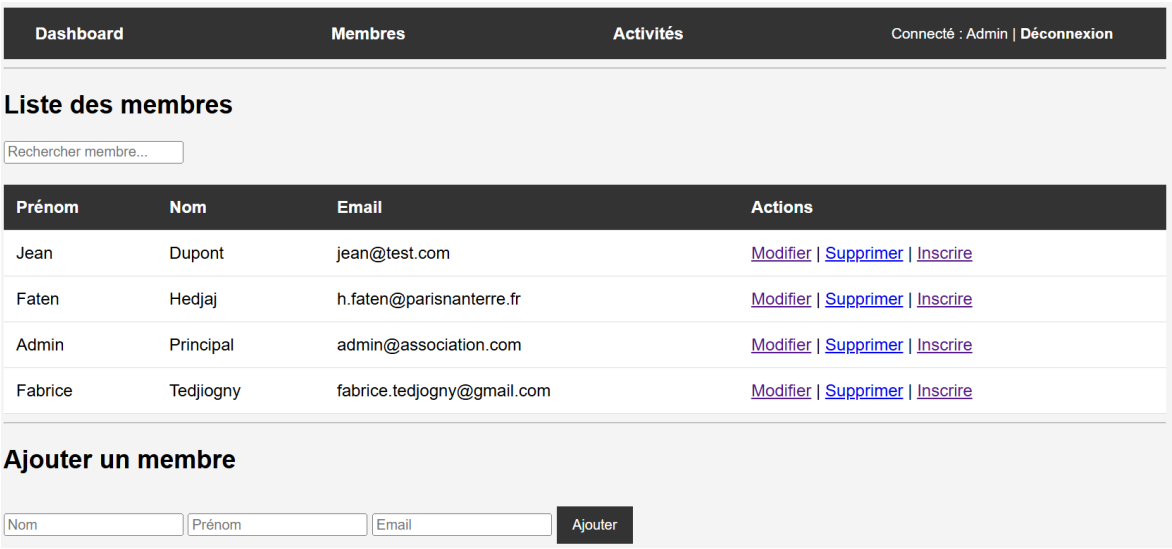

FIGURE 3 – Interface de gestion des membres

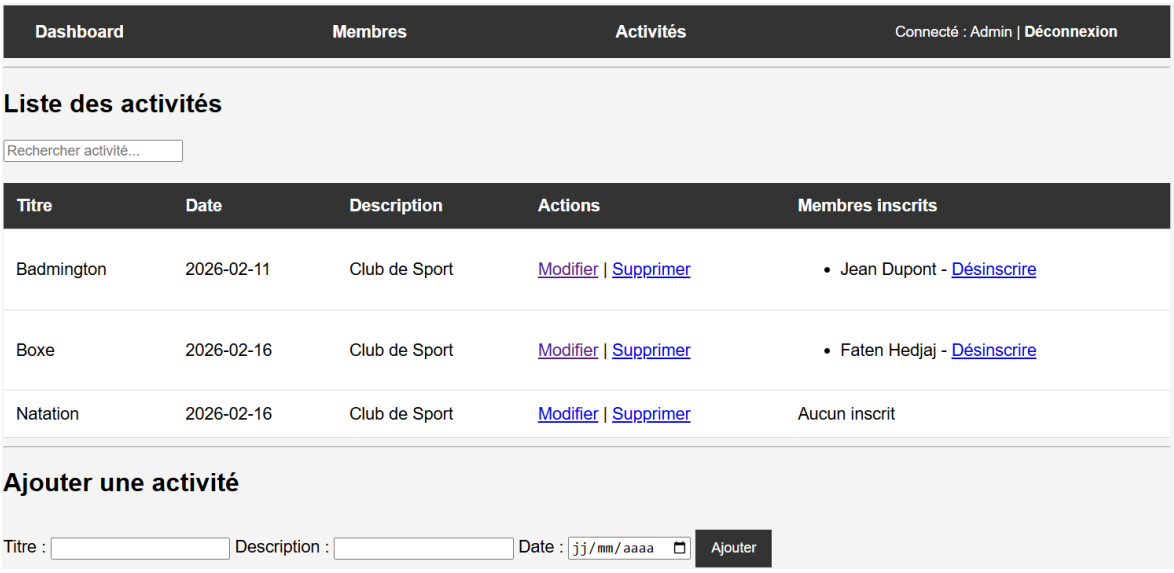

FIGURE 4 – Interface de gestion des activités

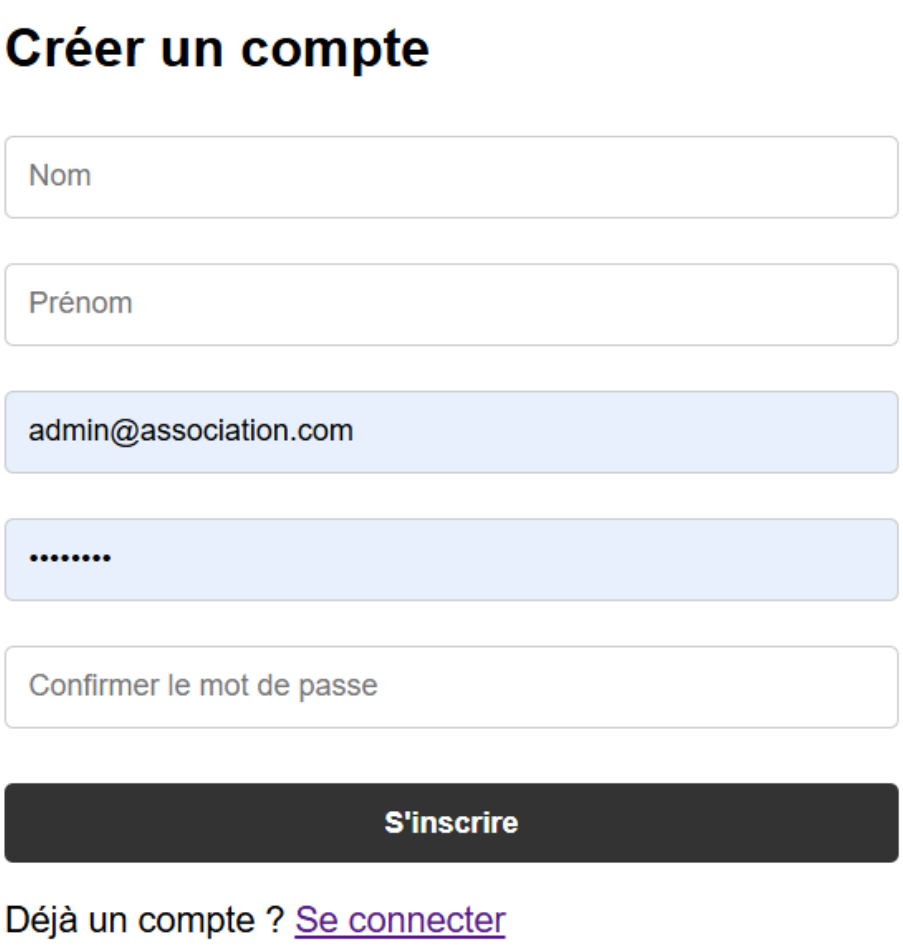

FIGURE 5 – Interface d'inscription d'un membre

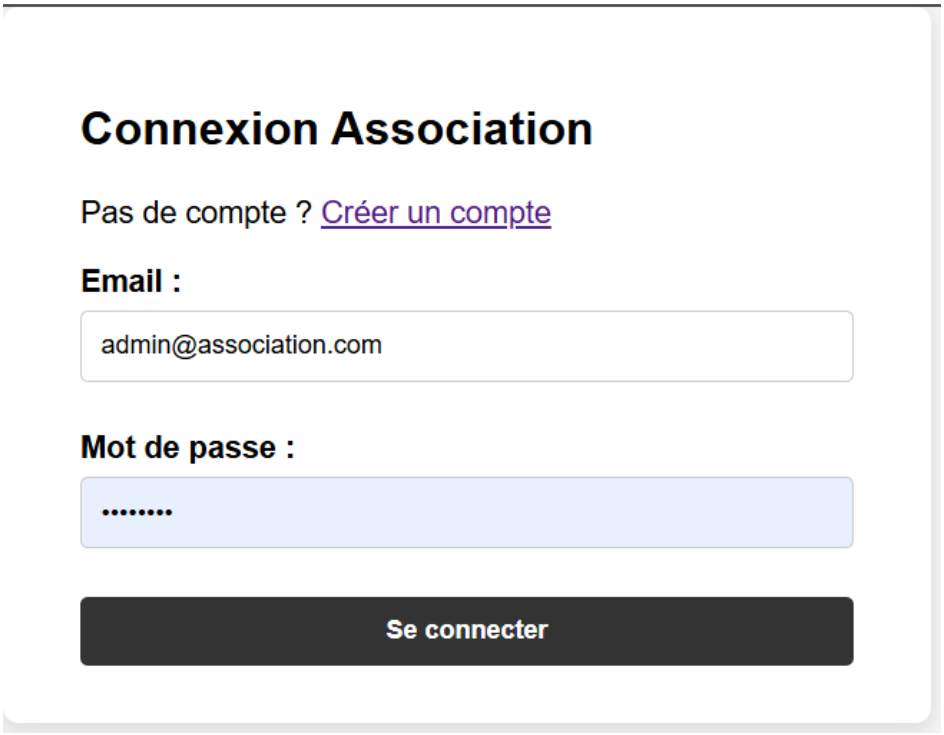

FIGURE 6 – Interface générale du site

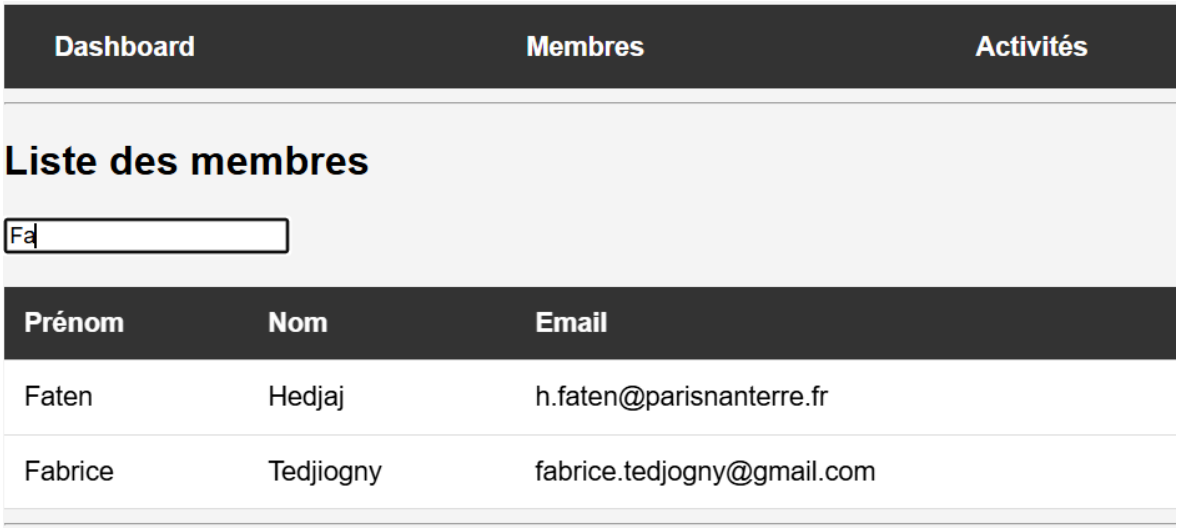

FIGURE 7 – Recherche dynamique en action

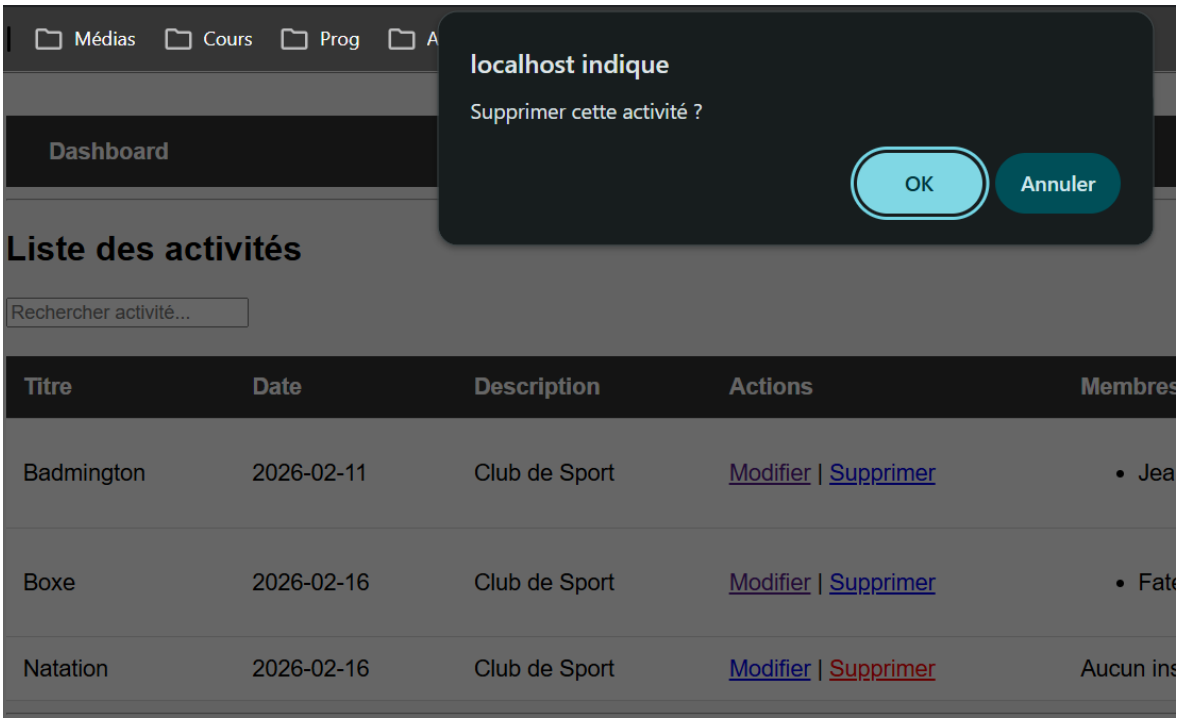

FIGURE 8 – Exemple de confirmation utilisateur